Lab Programs: Week 5

1. Write a JavaScript program to validate a 10-digit mobile number that starts with 6, 7, 8, or 9, using regular expressions.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Validate Mobile Number</title>
</head>
<body>
    <script>
        let mobile = prompt("Enter your 10-digit mobile number:");
        let pattern = /^[6-9]\d{9}$/;

        document.write(`Mobile number entered: ${mobile}<br>`);
        if (pattern.test(mobile)) {
            document.write("Valid mobile number");
        } else {
            document.write("Invalid mobile number");
        }
    </script>
</body>
</html>
```

Output:

Mobile number entered: 9876543210
Valid mobile number

2. Write a JavaScript program using callback functions to perform addition, subtraction, multiplication and division of two numbers based on the operator chosen by the user.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Calculator using Callback</title>
</head>
<body>
    <script>
        function add(a, b, callback) { callback(a + b); }
        function subtract(a, b, callback) { callback(a - b); }
        function multiply(a, b, callback) { callback(a * b); }
        function divide(a, b, callback) { callback(a / b); }

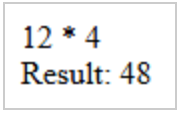
        function showResult(result) {
            document.write(`Result: ${result}`);
        }

        let num1 = Number(prompt("Enter first number:"));
        let num2 = Number(prompt("Enter second number:"));
        let operator = prompt("Choose operator (+, -, *, /):");

        document.write(`${num1} ${operator} ${num2}<br>`);

        if (operator === "+") {
            add(num1, num2, showResult);
        } else if (operator === "-") {
            subtract(num1, num2, showResult);
        } else if (operator === "*") {
            multiply(num1, num2, showResult);
        } else if (operator === "/") {
            divide(num1, num2, showResult);
        } else {
            document.write("Invalid operator");
        }
    </script>
</body>
</html>
```

Output:



3. Write a JavaScript program using Promises to simulate three sequential asynchronous tasks — "Order Placed", "Order Shipped", "Order Delivered" — printing each message only after the previous one completes, using .then() chaining.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Order Tracking using Promises</title>
</head>
<body>
    <div id="output"></div>
    <script>
        let output = document.getElementById("output");

        function step(message) {
            return new Promise((resolve) => {
                setTimeout(() => {
                    output.innerHTML += message + "<br>";
                    resolve();
                }, 1000);
            });
        }

        step("Order Placed")
            .then(() => step("Order Shipped"))
            .then(() => step("Order Delivered"));
    </script>
</body>
</html>
```

Output:



4. Rewrite the above sequential order-tracking simulation using async/await instead of .then() chaining, and compare the readability of both approaches in a comment.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Order Tracking using Async/Await</title>
</head>
<body>
    <div id="output"></div>
    <script>
        let output = document.getElementById("output");

        function step(message) {
            return new Promise((resolve) => {
                setTimeout(() => {
                    output.innerHTML += message + "<br>";
                    resolve();
                }, 1000);
            });
        }

        async function trackOrder() {
            await step("Order Placed");
            await step("Order Shipped");
            await step("Order Delivered");
        }

        trackOrder();

        // Comparison:
        // The .then() chaining version (previous program) works correctly, but each
        // step is nested inside a callback, so the sequence of events has to be
        // read "sideways" through chained .then() calls.
        // The async/await version above reads top-to-bottom like normal synchronous
        // code - each "await step(...)" line simply pauses until that step finishes
        // before moving to the next line. This makes the async/await version easier
        // to read and easier to extend (e.g. adding a try/catch for errors) compared
        // to the .then() chaining version.
    </script>
</body>
</html>
```

Output:



5. Write a JavaScript program to register mouseover, mouseout, and click event handlers on an image, displaying an appropriate message in each case.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Image Event Handlers</title>
    <style>
        img {
            width: 150px;
            height: 150px;
            border: 2px solid #333;
            cursor: pointer;
        }
        #message {
            margin-top: 10px;
            font-family: Arial, sans-serif;
            font-size: 16px;
        }
    </style>
</head>
<body>
    <img id="myImage"
         src="data:image/svg+xml;utf8,<svg xmlns='http://www.w3.org/2000/svg' width='150' height='150'>
<rect width='150' height='150' fill='%23ff7f50'/><text x='50%' y='50%' font-size='18' fill='white' text-
anchor='middle' dy='.3em'>Image</text></svg>"
         alt="Sample Image">
    <p id="message">Hover, move away, or click the image</p>

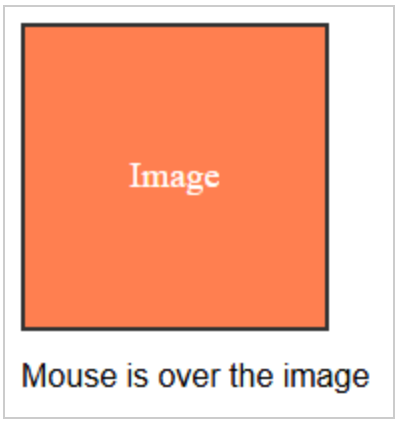
    <script>
        let img = document.getElementById("myImage");
        let message = document.getElementById("message");

        img.addEventListener("mouseover", function () {
            message.textContent = "Mouse is over the image";
        });

        img.addEventListener("mouseout", function () {
            message.textContent = "Mouse has left the image";
        });

        img.addEventListener("click", function () {
            message.textContent = "Image was clicked";
        });
    </script>
</body>
</html>
```

Output:



6. Write a JavaScript program using arrow functions along with the array methods map, filter, and reduce to process a list of student marks, and display the class average and the list of students who passed (marks >= 40).

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Student Marks - map, filter, reduce</title>
</head>
<body>
    <script>
        let students = [
            { name: 'A', marks: 35 },
            { name: 'B', marks: 78 },
            { name: 'C', marks: 42 },
            { name: 'D', marks: 90 }
        ];

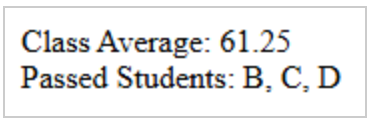
        let marksList = students.map(s => s.marks);

        let classAverage = marksList.reduce((sum, m) => sum + m, 0) / marksList.length;

        let passedStudents = students.filter(s => s.marks >= 40).map(s => s.name);

        document.write(`Class Average: ${classAverage.toFixed(2)}<br>`);
        document.write(`Passed Students: ${passedStudents.join(", ")}`);
    </script>
</body>
</html>
```

Output:



7. Write a JavaScript program to simulate a simple traffic light system (Red -> Green -> Yellow -> Red) using callback functions and setTimeout, allowing the user to start and stop the sequence using Start and Stop buttons and their click event handlers.

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Traffic Light Simulation</title>
    <style>
        #light {
            width: 100px;
            height: 100px;
            border-radius: 50%;
            background-color: red;
            margin-bottom: 10px;
            border: 2px solid black;
        }
        button {
            margin-right: 10px;
            padding: 6px 14px;
        }
    </style>
</head>
<body>
    <div id="light"></div>
    <button id="startBtn">Start</button>
    <button id="stopBtn">Stop</button>
    <p id="status">Status: Stopped</p>

    <script>
        let light = document.getElementById("light");
        let status = document.getElementById("status");
        let timer = null;
        let running = false;

        function changeLight(color, next, delay) {
            light.style.backgroundColor = color;
            status.textContent = `Status: ${color.toUpperCase()}`;
            timer = setTimeout(() => {
                if (running) next();
            }, delay);
        }

        function redLight() { changeLight("red", greenLight, 2000); }
        function greenLight() { changeLight("green", yellowLight, 2000); }
        function yellowLight() { changeLight("yellow", redLight, 1000); }

        document.getElementById("startBtn").addEventListener("click", function () {
            if (!running) {
                running = true;
                redLight();
            }
        });
```

```
        document.getElementById("stopBtn").addEventListener("click", function () {
            running = false;
            clearTimeout(timer);
            status.textContent = "Status: Stopped";
        });
    </script>
</body>
</html>
```

Output:

